Multi-Armed Bandit LLMs for Solving Mathematics

Ahmad Alsharif, Omar Alzoubi, Yazeed Alsaedi, Soumaya Chaffar

Department of Artificial Intelligence, University of Prince Mugrin, Saudi Arabia.

*Corresponding author(s). E-mail(s): s.chaffar@upm.edu.sa;
Contributing authors: 4110623@upm.edu.sa; 4110427@upm.edu.sa; 4110500@upm.edu.sa;

Abstract

We propose a dual-LLM framework for mathematical problem solving, pairing an open-source solver (LLaMA) with an external validator (GPT-4). To address the limitations of fixed prompting strategies, we introduce a multi-armed bandit agent that dynamically selects among four prompt styles—decomposition, analogy, formal proof, and backward solving—based on the domain of the problem. The system adapts over time to identify which strategy is most effective for domains such as algebra, geometry, and calculus.

We evaluate the approach on a stratified set of 50 problems from the GSM8K benchmark. Compared to a fixed chain-of-thought baseline, our method improves accuracy by 8 percentage points while reducing average latency by nearly half. The most substantial gains are observed in algebra and geometry.

This study provides evidence that combining dynamic prompt engineering, domain classification, and strategy optimization leads to measurable improvements in both correctness and efficiency. Beyond immediate performance gains, our system outlines a pathway toward more adaptive and modular math reasoning pipelines built on open-source foundations.

Keywords: Large Language Models, Multi-Armed Bandits, Mathematical Problem Solving, Prompt Engineering, Chain-of-Thought Reasoning

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable proficiency in Natural Language understanding and generation, yet multi-step mathematical reasoning remains a challenging frontier. Open-source models such as LLaMA may produce solutions that appear coherent but harbor hidden logical or arithmetic inconsistencies, especially in multi-step reasoning. For educational, professional, and research applications, ensuring rigor and correctness is paramount.

While GPT-4 demonstrates strong performance in solving a variety of math problems, it remains a closed-source model with usage restrictions that limit its scalability and adaptability. In contrast, open-source models such as LLaMA provide fine-tuning capabilities for domain-specific requirements and **are** capable of being self-hosted, with no dependency on proprietary services. In this work, we leverage both: **LLaMA** as the primary **solver**, and **GPT-4** as an external **validator**, verifying the logical consistency and correctness of the generated solutions.

Furthermore, introducing this external validation in combination with a bandit-based system improves a static LLaMA baseline that uses a single-pass chain-of-thought prompt.

This work answers three selected research questions: Can incorporating a multi-armed bandit agent with an LLM enhance accuracy in solving mathematical problems when compared to one fixed prompting strategy? Whether overall accuracy of having a choice of prompt strategies which are selected on-the-fly will be significantly higher than that of the one fixed prompt strategy? What is the performance gap between bandit-based adaptive prompt selection and classical chain-of-thought prompting when it comes to accuracy as well as computing speed? Our hypothesis is that the overall accuracy should be higher with the dynamic prompting selected by a multi-armed bandit exploration algorithm as compared with more static designs.

Dual-LLM setups have emerged as a promising direction, where one LLM acts as a solver and another LLM serves as a validator or "critic" that identifies flaws in the solver's output. In parallel, prompt engineering research shows that different prompting "styles" or "strategies"—ranging from formal proofs to decomposition—can significantly impact outcomes in math problem solving. We propose uniting these ideas within a multi-armed bandit system, in which each prompt strategy is treated as an "arm" and the system learns over time which strategy best suits each math domain. This paper outlines our literature review, the methodological foundation for the idea, and a conceptual pipeline that illustrates how problem classification, strategy selection, solver generation, and validator feedback interoperate within an adaptive loop.

2 Related Work

2.1 Large Language Models for Mathematical Reasoning

Recently, Large Language Models like GPT-4 [1], PaLM [15], and Minerva [18] have achieved remarkable success in text generation, reasoning, and problem-solving. However, open-source models like LLaMA [2], while offering advantages in customization

and deployment flexibility, often exhibit inconsistencies in complex mathematical reasoning tasks requiring multiple steps. These models may produce derivations that appear logically coherent but contain subtle errors, particularly when using fixed prompting strategies that may not be optimal for all mathematical domains. Early works by Saxton et al. [3] and Hendrycks et al. [4] highlight the challenges of multi-step reasoning in math for standard neural architectures, and demonstrate how increasingly complex or prooflike tasks compound these issues.

Chain-of-thought prompting [5] addresses this with an approach to increase the model's correctness in a step-by-step fashion by asking the model to write out intermediate steps. Yet, Wang et al. show that the output solutions may still remain inconsistent [6]. Methods such as self-consistency sample multiple chain-of-thought paths and choose the best final answer to increase accuracy, but fail to ensure reliability across all symbolic or step-by-step tasks.

Recent work by Cobbe et al. [16] proposed a training framework for verifiers where verifiers are trained to evaluate the correctness of mathematical solutions provided by language models. This is similar in spirit to the dual-LLM pipeline we use, where the validator LLM tries to catch mistakes in outputs generated by the solver. However, unlike Cobbe et al., our system does not rely on a fine-tuned verifier; instead, it uses GPT-4 in combination with a Multi-Armed Bandit Agent that adaptively selects the most effective prompting strategy based on problem domain. Additionally, Zelikman et al. [17] propose STaR (Self-Taught Reasoner) which incrementally bootstraps reasoning by iterating on model-generated solutions, shows how repeated verification can lead to considerable improvements in accuracy. These findings support our focus on an iteratively refined validator LLM to increase the reliability of multi-step mathematical reasoning.

Hybrid or coded solutions such as generating Python snippets for validation [19] are good at catching numeric errors but often miss more abstract reasoning. These approaches emphasize the more general issue: a single pass by a single LLM rarely ensures thorough correctness, especially for nontrivial math derivations. A critical analysis of existing approaches reveals three fundamental limitations: (1) static prompting strategies that fail to adapt to problem characteristics, (2) single-model architectures lacking independent verification mechanisms, and (3) absence of learning systems that can improve strategy selection over time. These gaps motivate our integration of adaptive prompt selection with validation frameworks. We note that also recently approaches adopt iterative self-feedback to improve LLM outputs in multi-step tasks. For example, an LLM can cycle between solution generation and critique [20], improving its output over successive passes. While this reflexive approach does not make use of a separate validator, it does highlight the importance of repeated checking closely related to the principle behind our separate solver–validator design. Experiments like Auto-GPT [21] also run agents whose primary responsibility is to actively plan, illustrate how minimum task-roles can work within (or across) LLMs to improve math-like reasoning by attempting to mitigate overlooked mistakes. Now, although these systems aren't precisely tuned into mathematics, they reinforce the overarching idea that structured feedback loops be it through a second LLM or some inner reflection mechanism can dramatically improve correctness in multi-step derivations.

2.2 Dual-LLM and Multi-Agent Architectures

To mitigate single-model issues, multi-agent architectures or dual-LLM pipelines are in development. One LLM (the solver) makes proposals and another LLM (the validator or critic) verifies correctness. Madaan et al. [7], showing iterative refining, where one model proposes a first solution and is critiqued by either the same or a different model Park et al. [8] demonstrate a setup where multiple "generative agents," each powered by a large language model, interact in a shared environment. While not restricted to a pair, this design exemplifies how multiple LLMs can cooperate, critique, or respond to one another's outputs in real time. Each agent is endowed with its own memory, goals, and internal states, leading to emergent social and reasoning behaviors that surpass what a single LLM might achieve in isolation.

In the context of theorem proving, Polu and Sutskever [9] introduced a generative language modeling system (GPT-f) for automated theorem proving in Metamath, bridging formal logic tasks with LLM-based text generation. However, while promising, these multi-agent approaches often rely on static prompt templates. They do not employ the prompting type (e.g., formal proof type vs. decomposition) by each specific math domain systematically. The next logical step is, therefore, why not create a specific per prompt and per domain which way of prompting to apply.

Outside of theorem proving, the recent frameworks adopt a "controller–executor" paradigm in which a centralized LLM delegates tasks to other specialized or auxiliary models. HuggingGPT [22] is representative of this approach, which leverages ChatGPT as a conductor to coordinate a slew of models hosted on Hugging Face to address heterogeneous AI tasks. Even though HuggingGPT isn't expressly for math, it illustrates how an agent, "chief," could delegate authority to many experts, a concept similar to how our pipeline harvested those into solver (Llama) and validator (GPT-4). Importantly, we include bandit-driven prompt selection for the solver, meaning that the solver–validator framework is allowed to change and adapt, rather than being constrained to some static means of forming problems.

2.3 Prompt Engineering and Strategy Variation

Different prompt engineering has a strong influence on math-solving capability. Reed and de Freitas [10], and Nye et al. [11] demonstrate that step-by-step or "scratchpad" prompts can improve the model's performance on intermediate calculations. Formal prompts come in handy for geometric or advanced calculus proofs, and decomposition prompts can be more efficient for multi-part integrals or simpler algebra. Analogy prompts could help take a new task and adapt it to one that is more well-formed, but there is the risk of partial mismatch when the analogy is misguided. These prompting strategies indicate no single technique is ideal for every math problem. How to evolve prompt selection over time, however, to match each problem's characteristics — an area that has garnered less attention at the intersection of multi-agent design and dynamic prompt engineering. Recent work in adaptive prompting, such as contextual-bandit systems for reading comprehension and dialogue generation has shown the importance of dynamically selecting templates. Our work is different in

that it addresses only mathematical reasoning, where the structure of prompts interacts heavily with domain-specific properties and an instance-specific adaptation can be applied in a pipeline of a solver and a validator.

Beyond solving math problems, there have been other approaches based on contextual bandits which choose per-instance among various prompt templates. Zhang et al. Reading comprehension tasks, for instance, have been found to improve when a bandit algorithm is used to adaptively select the best prompt based on live feedback [23]. Liu et al. [24] applied this concept to dialogue systems and showed that adaptive prompt switching can substantially improve dialogue quality. These results highlight the fact that no style of prompting rules them all, motivating our strategy of online exploration among four different prompts (decomposition, analogy, formal, backward solving) to figure out which is the one that works best in the given math domain.

2.4 Adaptive Strategy Selection: Multi-Armed Bandits

Multi-armed bandit (MAB) algorithms provide a general framework for adaptive action selection. Sutton and Barto [12] and Lattimore and Szepesvári [13] explain how an agent can incrementally learn which "arm" has the highest reward. In the LLM context, each "arm" represents a prompt strategy (formal, analogy, decomposition, heuristic), and the reward is the correctness measure that can be obtained from a validator. Chang et al. [14] highlight adaptively composing and selecting prompts can lead to systematic improvement in performance foreach of the tasks, especially when the tasks are heterogeneous with no single prompt style working best for all tasks. An epsilon-greedy strategy allows the system to continue to explore other strategies with probability ε, and exploit the best known strategy otherwise. Through repeated trials, the system home in on whatever strategies–domain pair provided the statistically best success rate, but still occasionally sample from other arms in order to avoid locally converging on poor maxima.

2.5 Symbolic and Hybrid Extensions

Some higher-level math tasks might be unwieldy under pure chain-of-thought prompting, and would therefore incentivize verification by way of symbolism or code-based checking. A dual-LLM pipeline could, in theory, include symbolic tools or code generation in the solver's chain-of-thought. While these approaches may be limited to specific domains, they highlight an unmet need for a role in an LLM pipeline that ensures a critical step of review after the solver has generated a partial or final solution, similar to the role of the validator LLM.

2.6 Research Gaps and Opportunities

Despite recent advances, three critical gaps remain in LLM-based mathematical reasoning research:

Single Pass Limitations: Even advanced chain-of-thought or self-consistency methods do not fully ensure correctness, lacking independent oversight.

Static Prompt Styles: can be a Rigged Strategy used on each, is not well-inclined to handle domains such as geometry, algebra, calculus.

Limited Use of Adaptive Policies: While the multi-armed bandit context is well-explored in Reinforcement Learning, picking up terms of service to adaptively request LLM prompts for math tasks is new territory.

With a combined two-LLM structure (solver + validator) and a diverse prompt strategy which includes a bandit-based approach to what prompt kind works best for each domain, we attempt to minimize errors made in single-pass (e.g. classifiers over address schemas) while improving outcomes through iterative feedback and task specification and domain specificity on outputs. This integration provides a new direction towards self improving, multi-agent LLM systems that use the lessons learned from each math problem to customize their own thinking style. Overall, multi-agent LLM setups for mathematical tasks have demonstrated value, but few systems methodically customize prompting dependent on mathematical domain through online learning. Works such as Reflexion [20], Auto-GPT [21], and HuggingGPT [22] illustrate powerful feedback between agents or iterations, yet usually assume static or user-given instructions for prompts. At the same time, bandit-based solutions in comprehension and conversation [23] validate the impact of adapting prompt selection, but do not delve into specialized, step-by-step mathematics. Our sequence unites a distinct solver (Llama) and validator (GPT-4) with domain-aware bandit direction of approaches—a synergy aimed at outperforming single-attempt Llama benchmarks in reliability and correctness.

3 Proposed Approach

3.1 Overview of the Proposed Architecture

We present a two-step dual-LLM pipeline, where one LLM (the solver) is given a math problem and generates a detailed, or a short answer, and another LLM (the validator) evaluates the correctness and logical coherence of the solution. A performance tracker logs wins and losses by strategy-domain pairing. The epsilon-greedy algorithm repeatedly chooses a strategy for problem domains based on the success rate of each one over time.

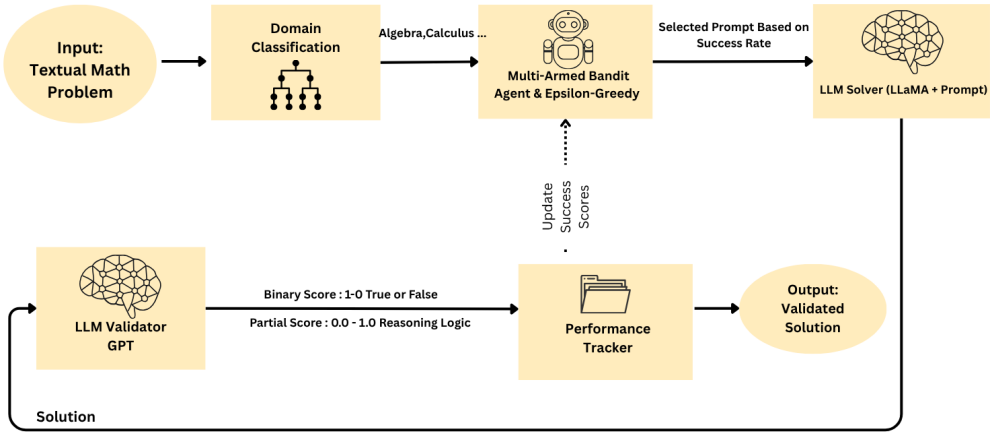

Fig. 1 End-to-end system design of the adaptive dual-LLM pipeline using LLaMA 3.2 3B Instruct and GPT-4.

The figure above presents a conceptual pipeline. It covers:

1. **Problem Classification (via GPT-4)**: The system uses GPT-4 to infer whether a problem falls under algebra, geometry, probability, number theory, or calculus.
2. **Multi-Armed Bandit Agent and Epsilon-Greedy**: The system picks one of the four prompt strategies (decomposition, analogy, formal proof, backward solving) based on historical performance, balancing exploitation of successful strategies with exploration of alternatives.
3. **LLM Solver**: The chosen prompt plus the original problem text is fed to the solver (e.g., Llama), which generates a proposed solution.
4. **LLM Validator**: GPT-4 then checks the solver's output for correctness, logic flow, and clarity, returning a correctness flag and a partial score between 0 and 1 that reflects logical validity.
5. **Performance Tracker**: The system updates the recorded success/failure count for the chosen strategy-domain combination, feeding that data back into the epsilon-greedy policy used by the Multi-Armed Bandit Agent for future strategy selection.

3.2 Multi-Armed Bandit Strategy Selection

3.2.1 Formal Setup.

Let $\mathcal{D}$ be the set of math domains (e.g., $\{\text{algebra}, \text{geometry}, \text{calculus}, \ldots\}$) and let $\mathcal{S} = \{1, 2, \ldots, K\}$ be the set of K possible prompt strategies (arms). For each domain $d \in \mathcal{D}$ and strategy $s \in \mathcal{S}$, we track a running estimate of its success rate or expected reward, denoted $Q_{d,s}$. At time step t, we receive a new problem belonging to domain d; we must select a strategy s (an "arm") to solve it.

We then observe a binary reward $r_t \in \{0, 1\}$ from the validator (GPT-4) that indicates correctness (1 if correct, 0 otherwise). We update the corresponding value estimate $Q_{d,s}$ to incorporate the new reward:

$$Q_{d,s} \leftarrow Q_{d,s} + \alpha[r_t - Q_{d,s}], \tag{1}$$

where α is a learning rate (which can be $1/N_{d,s}$ if you prefer the standard average-based update, or a small fixed constant). If you use an average of all observed rewards for that domain–strategy pair, then:

$$Q_{d,s} = \frac{1}{|T_{d,s}|} \sum_{t \in T_{d,s}} r_t, \tag{2}$$

where $T_{d,s}$ is the set of time steps in which domain d used strategy s.

3.2.2 Epsilon-Greedy Action Selection.

To balance *exploration* (trying each strategy enough to discover its true potential) with *exploitation* (using the best-known strategy so far), we employ an ε-greedy policy [12]. Concretely, if we are given a new problem from domain d:

- Exploration: With probability ε, choose a random strategy $s \in \mathcal{S}$.
- Exploitation: With probability $1 - \varepsilon$, choose the strategy $s^* = \arg\max_{s \in \mathcal{S}} Q_{d,s}$.

Equivalently, the probability of choosing strategy s at time t is often described as:

$$\pi_t(s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{S}|} & \text{if } s = \arg\max_{s' \in \mathcal{S}} Q_{d,s'} \\ \frac{\varepsilon}{|\mathcal{S}|} & \text{otherwise} \end{cases} \tag{3}$$

This ensures that you most often pick the best-estimated arm, but *sometimes* (with probability ε) explore other strategies. Over repeated trials, the method adapts to each domain, converging to whichever prompt style achieves the highest success rate.

3.2.3 Epsilon Decay.

Because we initially want more exploration, ε often starts at a moderate value (e.g., 0.3) and decays over time. One simple decay rule is:

$$\varepsilon_t = \max(\varepsilon_{\min}, \beta \varepsilon_{t-1}), \tag{4}$$

where $0 < \beta < 1$ is a decay factor (e.g., $\beta = 0.99$), and $\varepsilon_{\min}$ is a floor value (e.g., 0.05) so the policy always retains some degree of exploration.

3.2.4 Reward and Updates by Domain.

Because domain d might influence the reward distribution, we maintain separate value estimates for each domain–strategy pair, i.e., $Q_{d,s}$. If the new problem is from geometry, we only update the geometry–strategy pair $Q_{\text{geometry},s}$. This domain separation ensures the system can discover that, for instance, formal proofs excel in geometry whereas decomposition might be better for calculus integrals.

3.3 Epsilon-Greedy Strategy

For each strategy–domain pair, we have an estimated success rate. So if "formal proof with geometry" has been a success 8 out of 10 times, we estimate its success rate at 80%. Given a new problem instance, we repeat the following: With probability ε (initially set to 0.1, following standard exploration practice [12]), choose a random strategy to explore.

With probability $1 - \varepsilon$, select the best known strategy for that domain (highest success rate so far).

The system tracks the success/failure of each strategy–domain pair and updates their estimated success rates accordingly, based on validator feedback. After multiple tries, ε is reduced to a small, close-to-minimum (for instance, 0.05) so that more exploitation of the best strategy is retained while randomly exploring remains limited.

3.4 Dataset

We evaluate our dual-LLM pipeline using the GSM8K dataset, a widely-used benchmark for grade-school mathematical reasoning [16]. The problems in the GSM8K cover algebra and probability, but lack sufficient coverage of geometry, number theory, and calculus, which limits their suitability for evaluating strategies across diverse mathematical domains. According to GSM8K, each problem has a structured solution, which can be directly verified by our secondary LLM. To fill in missing domains and increase variety, we include synthetic problems generated by GPT-4. These synthetic questions are designed to fill gaps in domain coverage and introduce variations in problem structure, enabling us to test the approach's adaptability across a broader range of formats.

For the synthetic problems we generated using GPT-4, we provided an ad hoc template for the domain that requested a 'Problem: ...' and 'Solution: ...' pair for each category (algebra, geometry, probability, number theory, and calculus). Each question comprised one system command ("You are an expert math problem generator") and one user query describing how the answer should be formatted. The responses were parsed by removing the 'Solution:' divider and extracting the problem statement and solution text. We manually excluded items that were ill-formed, ambiguous, or incorrectly formatted. These synthetic tasks were exclusively for ensuring balanced domain representation within the dataset, rather than to serve as substitutes for any part of the GSM8K difficulty or evaluation workflow.

While these synthetic items broaden domain coverage, they are intentionally simpler than GSM8K's multi-step problems because each one required GPT-4-based generation and verification.

Each problem is first categorized by GPT-4, which infers the domain from the problem text, and then the problems are passed into the **Multi-Armed Bandit Agent**. To prevent overfitting of our model to one type of reasoning problem, the dataset is **balanced** across different mathematical topics. The purpose of the dataset is primarily to test the adaptive epsilon-greedy strategy, where our system gradually enhances its ability to solve problems by improving its strategy selection over time.

Our experimental setup uses a stratified sample of the GSM8K dataset as we are limited by resources required to query. Since every problem entails multiple interactions with GPT-4 to classify its domain, verify the generated solution, and score it, scaling to the entire GSM8K set goes beyond our current API rate limits as well as our budget. To keep the evaluation practical and replicable in these conditions, we chose a balanced blend of questions from algebra, geometry, probability, number theory and calculus. Although the dataset size is modest, with our controlled setting we can isolate the effect of adaptive prompt selection under realistic resource constraints.

3.5 Baseline Comparison

We benchmark our bandit-enhanced system against single-LLM baseline, which has a hard-coded chain-of-thought prompt. The two systems will use GPT-4, as a third party validator to indicate correctness and quality of reasoning. But the main distinction is in the fact that we apply a Multi-Armed Bandit Agent which actively chooses the most efficient prompting strategy to use each problem domain, according to the past performance. This will allow the system to learn with time what to do with various categories of math problems.

3.6 Evaluation Metrics

We evaluate the system using the following three core metrics:

- **Correctness**: A Boolean or numeric indicator of whether the final answer matches the ground truth, as determined by GPT-4, which returns a correctness flag based on semantic and numerical agreement with the reference solution.
- **Step Validity**: A partial score in the range [0, 1], returned by GPT-4, reflecting the logical consistency and coherence of the reasoning steps—even if the final answer is incorrect.
- **Latency**: The wall-clock time measured from the beginning of solver execution to the receipt of the validator's judgment.

We additionally track the system's exploration–exploitation behavior during training by logging the proportion of problems assigned to each strategy. This allows us to verify that ε decays as intended and that the Multi-Armed Bandit Agent progressively shifts from exploration to exploiting the best-performing strategies.

3.7 Implementation Details

Python was used in our system with libraries and setting parameters. The solver element makes use of the Hugging Face Transformers library to load the local LLaMA 3.2-3B-Instruct (meta-llama/Llama-3.2-3B-Instruct) using device mapping that is device-accelerated with a GPU. Specifically, it prompts the model with temperature=0.7, truncation turned off and max tokens=500. The validator component takes the OpenAI GPT-4 API with temperature=0.3 and max tokens=300 for consistency evaluation.

We intentionally use LLaMA-3.2-3B-Instruct because one of the key research questions we seek to answer is how adaptive prompting can help with mathematical

reasoning in small models without benefitting from the capacity gains that larger architectures can provide.

The epsilon-greedy exploration used includes the following parameters of multi-armed bandit implementation: initial epsilon=0.1, decay factor=0.99, and minimum epsilon=0.05. Four different prompting methods were applied, namely, decomposition (step-by-step strategic plan), analogy (strategic plan based on patterns), formal proof (strategic plan based on mathematical proofs) and backward solving (strategic plan based on achieving a goal). GPT-4 with temperature=0.0 and max tokens=10 are used to classify problems in terms of their domains.

We selected these epsilon-greedy parameters following standard practice, and preliminary tests indicated that the system behaved stably under small variations of these values.

Training was conducted using 100 problems and evaluation measured using 50 problems. GSM8K was used as a foundation with the help of GPT-4 synthetic problems generated to balance the domain coverage. The experiments were all done on the OpenAI API to validate and classify, and locally through LLaMA inference of the solution generation. Details of all four strategies are shown in Table 1 below.

Table 1 Prompt templates used for each strategy

Strategy	**Prompt Template**
Decomposition	Solve step by step: {`problem`}. Show your work clearly.
Analogy	Solve this problem: {`problem`}. Think of similar problems and apply that approach.
Formal Proof	Solve this rigorously: {`problem`}. Use formal mathematical methods.
Backward Solving	Solve by working backwards: {`problem`}. Start from what you need to find.

The four prompt templates were chosen so that each one represents a different type of approach that students generally perform when reasoning mathematically. The decomposition prompt is designed to encourage the solver to decompose a task into more manageable steps, and the analogy prompt is meant to help them to remember a similar problem and adjust their solution. The proof strategy prompt follows a more traditional, logically sequenced style, while the backward-reasoning template starts with a goal state and works backward. Collectively, these templates represent a diverse set of reasoning habits that the bandit may switch between based on an instance.

4 Evaluation and Results

4.1 Evaluation Protocol

We evaluated the Bandit-enhanced system against a strong fixed Chain-of-Thought (CoT) baseline, using a stratified set of 50 GSM8K test problems (10 per domain), with both systems relying on GPT-4 for final validation and scoring. For each problem, we recorded three core metrics—**Correctness**, **Partial Score**, and **Latency**—as defined in Section 3.5. Both agents used GPT-4 to evaluate correctness and reasoning quality.

4.2 Overall Performance

Figure 2 illustrates the success-rate breakdown by domain for both the Baseline and our Bandit-enhanced agent. We observe that:

- Algebra increases from 50% to 70% (+20 pp)
- Geometry increases from 10% to 40% (+30 pp)
- Calculus increases from 30% to 40% (+10 pp)
- Number Theory remains at 30%
- Probability decreases from 60% to 40% (−20 pp)

These results confirm that our adaptive prompt-selection strategy yields the largest improvements in algebra and geometry, with modest gains in calculus. Number theory remains a difficult domain, and probability performance declined slightly. The decline in probability performance (-20pp) suggests that our current strategy portfolio may not be optimal for combinatorial reasoning tasks that require systematic enumeration of outcomes, indicating an area for future strategy refinement.

The lower performance in the probability domain is due to the structure of problems, for which one usually needs multi-branch case analysis or conditional reasoning. These moves also do not perfectly fit into our existing prompt templates, which are more directly geared towards algebraic manipulation or sequences of deterministic reasoning steps. In a number of other examples, the solver gave partial reasoning that was accepted by the validator (but still did not account for all probabilistic possibilities). Addressing this will require constructing new templates tailored for probability tasks, as well as better validation procedures.

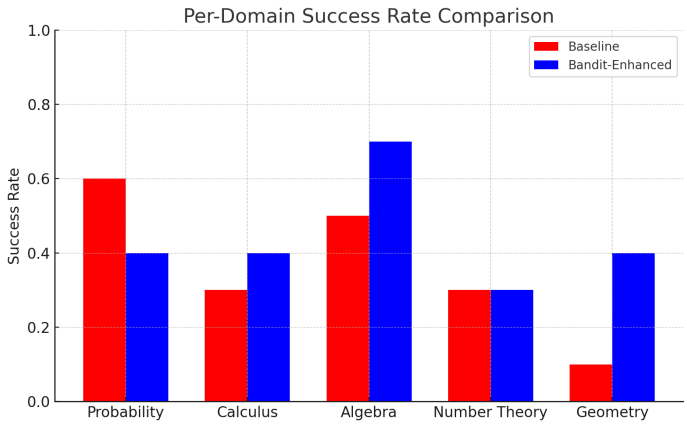

Fig. 2 Per-domain success rate comparison between Baseline and Bandit system based on GPT-4 validation.

The absolute gains in accuracy are modest, but shows that even small models such as LLaMA-3B can benefit from adaptive prompting and the primary benefit of the approach is the marked reduction in response latency and increased consistency across domains.

Table 2 shows the overall performance comparison.

Table 2 Performance on 50 GSM8K Problems

Agent	Accuracy	Mean Score	Latency (s)
Baseline	36% (18/50)	0.806	278 ± 16
Bandit (ours)	44% (22/50)	0.830	147 ± 12

Across the incorrect cases, we found specialized error patterns. Mistakes in algebra frequently resulted from arithmetic errors and missing sub-steps of multi-stage derivations. Failures of probability often resulted from the partial enumeration of cases or the misapplication of conditional laws. Geometry errors typically lacked necessary justification steps. Specific patterns for strategies emerged, For example, decomposition reduced arithmetic errors, backward reasoning supported goal-oriented tasks, and analogy was ineffective because of weak structural alignment with most problem types.

4.3 Latency and Token Efficiency

The baseline CoT prompt produces approximately 450 tokens per answer (nearly hitting the 500-token cap), whereas domain-tailored templates average approximately 180 tokens. This verbosity is the primary cause of the 2× latency gap.

The GPT-4 domain-classification step adds a small overhead of approximately 1–5 seconds per query, which is minor compared to the total solver latency and does not materially affect the comparative results.

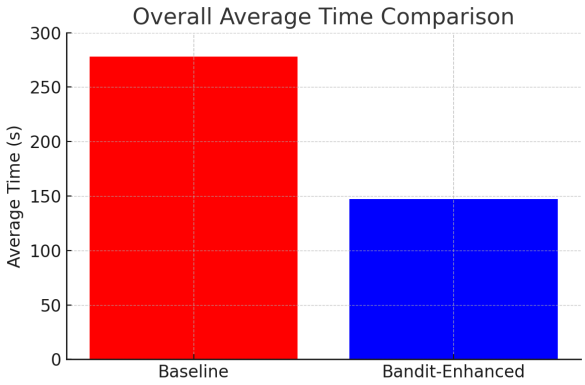

Fig. 3 Wall-clock latency per problem for Baseline vs. Bandit agents.

As shown in Fig. 3, our bandit-enhanced agent reduces average per-problem latency from **278 s to 147 s**. The GPT-4 domain-classification step adds a small overhead of approximately 1–2 seconds per query, which is minor compared to the total solver latency and does not materially affect the comparative results.

4.4 Bandit Convergence

During the 100-problem training phase, ε decayed from 0.10 to 0.06, and per-strategy success rates converged to *decomposition* 0.73, *backward solving* 0.64, *formal proof* 0.57, *analogy* 0.40, explaining the template mix at test time.

5 Discussion

The experiment confirms that combining prompt diversity with online strategy selection can outperform a fixed chain-of-thought prompt, even when using identical model weights. Accuracy gains arise from the system's ability to adaptively match prompt strategies to specific problem domains; The observed latency reduction stems from the use of domain-specific prompt templates, which produce more concise completions. Limitations include dependence on GPT-4 for validation, the small size of the evaluation set, and the absence of symbolic verification for algebraic reasoning.

5.1 Results Interpretation and Field Implications

The experiment affirms that prompt diversification can be combined with online strategy selection to yield better results than a fixed chain-of-thought prompt, on the same model weights. Improved accuracy is the result of the process of the system using an adaptive model of matching prompt strategies to particular problems domains; The latency reduction observed is the result of domain-specific prompt templates leading to shorter completions. The drawbacks are forward thinking on GPT-4 verification, tiny volume of evaluation pool, and symbolic validation of algebraic derivation.

The work is a combination of prompt engineering, multi-armed bandit algorithms and the design of a two LLM system into a pipeline for solving mathematical problems. Based on a solver–validator LLM pair and evaluating the performance per strategy–domain pair, the system can dynamically change its course of action, and minimize the chances to miss or output incorrect solutions. The design particularly fits subjects involving step-by-step reasoning, e.g., calculus or a geometric proof. There are other extensions to these simpler arrangements, such as including more sophisticated symbolic math modules, a wider variety of more domain types, or more sophisticated forms of the bandits (such as UCB or Thompson sampling) which could contribute to choice of strategy.

There are several limitations and failure cases in our method as well. We noted that certain types of problems were not well handled by the prompt strategies, in particular probability with case enumeration or conditional reasoning. Given the small training set, a bandit system can also converge toward suboptimal strategies, and in some cases, the solver may output inconsistent steps which are nevertheless partially correct according to the validator. These aspects underscore the importance of richer prompt templates and more powerful symbolic or programmatic verification in subsequent versions of this system.

We are currently only using GPT-4 as an external validator. Although GPT-4 shows good common-sense reasoning and consistency checks, it is not open source, so reproducibility is a concern, and there may be a bias when comparing solvers

with this system. We chose to use GPT-4 based on its proven track record for early-stage prototyping, but we understand that this is a limitation. We sketch in our full version an approach to substitute GPT-4 with a fine-tuned lightweight open-source verifier trained on graded solutions, which eliminates the closed-source dependency and enables substantially larger scale evaluation.

We note that a full ablation specifically isolating the bandit's contribution was not conducted given compute and API constraints. A controlled ablation is planned for the extended version of this work.

5.2 Strategy Behavior Across Domains

Examination of the strategy update logs suggests that the bandit converges toward stable policies that generalize across domains. Decomposition proves the most reliably successful template, especially for smaller models like LLaMA-3B that benefit from explicit step structuring and reduced cognitive load. Its success rates improve over those of baseline models for increasingly complex algebra, calculus, number theory and probability problems, indicating that detailed step-by-step guidance is particularly useful at this difficulty level.

Other strategies remain useful but are more situational. Backward-solving is often useful for tasks in calculus and algebra, where there is an easily recognizable target expression to work backwards from, and formal-proof prompting is helpful in geometry and number-theory items that require highly structured logical justification. The analogy prompting produces high quality solutions, but it is less reliable overall, so it is wise for the bandit to decrease its use in favour of these prompts that have more stable performance.

5.3 Future Directions

Future work will (1) run exact-match evaluation on the full GSM8K split, (2) replace GPT-4 with a lightweight verifier fine-tuned on graded solutions, and (3) explore Thompson-sampling bandits to cut early exploration cost.

We discuss future work that will (1) perform exact-match evaluation against the entire 8K GSM split, (2) scale to the alternative of using a lightweight verifier fine-tuned on graded solutions to replace GPT-4 and (3) investigate the use of Thompson-sampling bandits to reduce the early cost of exploration.

Lastly, when the Multi-Armed Bandit Agent has come into the strategies that are effective in each domain, dependence on GPT-4 in validating this strategy can be eliminated. It may be substituted by a lightweight verifier so that the system is fully based on open-source tools, but still performs well compared to static prompting baselines. A more comprehensive sensitivity analysis of these parameters is planned for future work.

To get beyond GPT-4, our next goal is to build a training corpus of solver outputs and associated correctness labels derived from a blend of automatic checks and human review. We propose to fine-tune a light-weight verifier model (1-3B parameters) on this dataset which can verify correctness of intermediate steps, as well as answers. If

successful, this verifier would entirely supplant the GPT-4 module and permit large-scale, completely open-source evaluation.

5.4 Ethical Considerations

We also highlight some ethical concerns. These include the potential overuse of automated reasoning tools in educational settings, the risk of spreading model errors or biases with incorrect validations, and the necessity for transparency of solver–validator interactions. It is important to find ways to help users distinguish between model-generated reasoning and sound mathematical proofs.

Acknowledgements

We would like to thank the University of Prince Mugrin for supporting this research.

Declarations

- **Funding**: This work was supported by the University of Prince Mugrin, Saudi Arabia.
- **Conflict of interest**: The authors declare no competing interests.
- **Ethics approval**: Not applicable.
- **Consent for publication**: All authors consent to publication.
- **Data availability**: The GSM8K dataset is publicly available. The synthetic problems generated for domain balancing are not publicly available.
- **Code availability**: Code is available upon request from the corresponding author.
- **Author contribution**: All authors contributed equally to this work.

References

[1] OpenAI: GPT-4 Technical Report. OpenAI (2023)

[2] Touvron, H., et al.: LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:2302.13971 (2023)

[3] Saxton, D., Grefenstette, E., Hill, F., Kohli, P.: Analysing Mathematical Reasoning Abilities of Neural Models. In: International Conference on Learning Representations (ICLR) (2019)

[4] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S.: Measuring Massive Multitask Language Understanding. In: International Conference on Learning Representations (ICLR) (2021)

[5] Wei, J., et al.: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv preprint arXiv:2201.11903 (2022)

[6] Wang, X., et al.: Self-Consistency Improves Chain-of-Thought Reasoning in Language Models. arXiv preprint arXiv:2203.11171 (2022)

[7] Madaan, A., et al.: Self-Refine: Iterative Refinement with LLMs. arXiv preprint arXiv:2303.17651 (2023)

[8] Park, J.S., et al.: Generative Agents: Interactive Simulacra of Human Behavior. arXiv preprint arXiv:2304.03442 (2023)

[9] Polu, S., Sutskever, I.: Generative Language Modeling for Automated Theorem Proving. arXiv preprint arXiv:2009.03393 (2020)

[10] Reed, S., de Freitas, N.: Neural Programmer-Interpreters. arXiv preprint arXiv:1511.06279 (2015)

[11] Nye, M., et al.: Show Your Work: Scratchpads for Intermediate Computation with Language Models. arXiv preprint arXiv:2112.00114 (2021)

[12] Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge (2018)

[13] Lattimore, T., Szepesvári, C.: Bandit Algorithms. Cambridge University Press, Cambridge (2020)

[14] Chang, K., Li, Y., Zhang, C.: Efficient Prompting Methods for Large Language Models: A Survey. arXiv preprint arXiv:2404.01077 (2024)

[15] Chowdhery, A., et al.: PaLM: Scaling Language Modeling with Pathways. Google AI Blog (2022)

[16] Cobbe, K., et al.: Training Verifiers to Solve Math Word Problems. arXiv preprint arXiv:2110.14168 (2021)

[17] Zelikman, E., Wu, Y., Wu, T., Goodman, N.: STaR: Bootstrapping Reasoning With Reasoning. arXiv preprint arXiv:2203.14465 (2022)

[18] Lewkowycz, A., et al.: Solving Quantitative Reasoning Problems with Language Models. arXiv preprint arXiv:2206.14858 (2022)

[19] Chen, M., et al.: Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374 (2021)

[20] Shinn, A., Labash, M., Gao, J.: Reflexion: An autonomous agent with dynamic memory and self-reflection. arXiv preprint arXiv:2303.11366 (2023)

[21] Significant Gravitas: Auto-GPT: An autonomous GPT-4 experiment. GitHub (2023)

[22] Shen, Y., Song, K., Tan, M., et al.: HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. arXiv preprint arXiv:2303.17580 (2023)

[23] Zhang, Y., Cai, Q., Xiong, C.: Contextual bandit for prompt selection in reading comprehension. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (2022)

[24] Liu, Y., Dong, Q., Li, H., Huang, M.: Adaptive prompt selection using bandits for dialogue systems. arXiv preprint arXiv:2301.01234 (2023)